Macos Specific Options

--argv-emulation Enable argv emulation for macOS app bundles. If enabled, the initial open document/URL event is processed by the bootloader and the passed file paths or URLs are appended to sys.argv.

--osx-bundle-identifier BUNDLE_IDENTIFIER macOS .app bundle identifier is used as the default unique program name for code signing purposes. The usual form is a hierarchical name in reverse DNS notation. For example: com.mycompany.department.appname (default: first script's basename)

--target-architecture, --target-arch ARCH Target architecture (macOS only; valid values: x86_64, arm64, universal2). Enables switching between universal2 and single-arch version of frozen application (provided python installation supports the target architecture). If not target architecture is not specified, the current running architecture is targeted.

--codesign-identity IDENTITY Code signing identity (macOS only). Use the provided identity to sign collected binaries and generated executable. If signing identity is not provided, ad- hoc signing is performed instead.

--osx-entitlements-file FILENAME Entitlements file to use when code-signing the collected binaries (macOS only).

Rarely Used Special Options

--runtime-tmpdir PATH Where to extract libraries and support files in *onefile* mode. If this option is given, the bootloader will ignore any temp-folder location defined by the runtime OS. The `_MEIxxxxxx`-folder will be created here. Please use this option only if you know what you are doing. Note that on POSIX systems, PyInstaller's bootloader does NOT perform shell-style environment variable expansion on the given path string. Therefore, using environment variables (e.g., ~ or `$HOME`) in path will NOT work.

--bootloader-ignore-signals Tell the bootloader to ignore signals rather than forwarding them to the child process. Useful in situations where for example a supervisor process signals both the bootloader and the child (e.g., via a process group) to avoid signalling the child twice.

ENVIRONMENT VARIABLES

PYINSTALLER_CONFIG_DIR
This changes the directory where PyInstaller caches some files. The default location for this is operating system dependent, but is typically a subdirectory of the home directory.

Windows And Macos Specific Options

-c, --console, --nowindowed Open a console window for standard i/o (default). On Windows this option has no effect if the first script is a '.pyw' file.

-w, --windowed, --noconsole Windows and macOS: do not provide a console window for standard i/o. On macOS this also triggers building a macOS .app bundle. On Windows this option is automatically set if the first script is a '.pyw' file. This option is ignored on *NIX systems.

–hide-console {minimize-early,hide-late,minimize-late,hide-early}

Windows only: in console-enabled executable, have bootloader automatically hide or minimize the console window if the program owns the console window (i.e., was not launched from an existing console window).

-i, --icon <FILE.ico or FILE.exe,ID or FILE.icns or Image or "NONE"> FILE.ico: apply the icon to a Windows executable. FILE.exe,ID: extract the icon with ID from an exe. FILE.icns: apply the icon to the .app bundle on macOS. If an image file is entered that isn't in the platform format (ico on Windows, icns on Mac), PyInstaller tries to use Pillow to translate the icon into the correct format (if Pillow is installed). Use "NONE" to not apply any icon, thereby making the OS show some default (default: apply PyInstaller's icon). This option can be used multiple times.

--disable-windowed-traceback Disable traceback dump of unhandled exception in windowed (noconsole) mode (Windows and macOS only), and instead display a message that this feature is disabled.

Windows Specific Options

--version-file FILE Add a version resource from FILE to the exe.

--manifest <FILE or XML> Add manifest FILE or XML to the exe.

-m <FILE or XML> Deprecated shorthand for –manifest.

-r, --resource RESOURCE Add or update a resource to a Windows executable. The RESOURCE is one to four items, FILE[,TYPE[,NAME[,LANGUAGE]]]. FILE can be a data file or an exe/dll. For data files, at least TYPE and NAME must be specified. LANGUAGE defaults to 0 or may be specified as wildcard * to update all resources of the given TYPE and NAME. For exe/dll files, all resources from FILE will be added/updated to the final executable if TYPE, NAME and LANGUAGE are omitted or specified as wildcard *. This option can be used multiple times.

--uac-admin Using this option creates a Manifest that will request elevation upon application start.

--uac-uiaccess Using this option allows an elevated application to work with Remote Desktop.

--collect-data, --collect-datas MODULENAME Collect all data from the specified package or module. This option can be used multiple times.

--collect-binaries MODULENAME Collect all binaries from the specified package or module. This option can be used multiple times.

--collect-all MODULENAME Collect all submodules, data files, and binaries from the specified package or module. This option can be used multiple times.

--copy-metadata PACKAGENAME Copy metadata for the specified package. This option can be used multiple times.

--recursive-copy-metadata PACKAGENAME Copy metadata for the specified package and all its dependencies. This option can be used multiple times.

--additional-hooks-dir HOOKSPATH An additional path to search for hooks. This option can be used multiple times.

--runtime-hook RUNTIME_HOOKS Path to a custom runtime hook file. A runtime hook is code that is bundled with the executable and is executed before any other code or module to set up special features of the runtime environment. This option can be used multiple times.

--exclude-module EXCLUDES Optional module or package (the Python name, not the path name) that will be ignored (as though it was not found). This option can be used multiple times.

--splash IMAGE_FILE (EXPERIMENTAL) Add an splash screen with the image IMAGE_FILE to the application. The splash screen can display progress updates while unpacking.

How To Generate

-d, –debug {all,imports,bootloader,noarchive}

Provide assistance with debugging a frozen application. This argument may be provided multiple times to select several of the following options. - all: All three of the following options. - imports: specify the -v option to the underlying Python interpreter, causing it to print a message each time a module is initialized, showing the place (filename or built-in module) from which it is loaded. See https://docs.python.org/3/using/cmdline.html#id4. - bootloader: tell the bootloader to issue progress messages while initializing and starting the bundled app. Used to diagnose problems with missing imports. - noarchive: instead of storing all frozen Python source files as an archive inside the resulting executable, store them as files in the resulting output directory.

--optimize LEVEL Bytecode optimization level used for collected python modules and scripts. For details, see the section "Bytecode Optimization Level" in PyInstaller manual.

--python-option PYTHON_OPTION Specify a command-line option to pass to the Python interpreter at runtime. Currently supports "v" (equivalent to "–debug imports"), "u", "W <warning control>", "X <xoption>", and "hash_seed=<value>". For details, see the section "Specifying Python Interpreter Options" in PyInstaller manual.

-s, --strip Apply a symbol-table strip to the executable and shared libs (not recommended for Windows)

--noupx Do not use UPX even if it is available (works differently between Windows and *nix)

--upx-exclude FILE Prevent a binary from being compressed when using upx. This is typically used if upx corrupts certain binaries during compression. FILE is the filename of the binary without path. This option can be used multiple times.

Options

-h, --help show this help message and exit

-v, --version Show program version info and exit.

--distpath DIR Where to put the bundled app (default: ./dist)

--workpath WORKPATH Where to put all the temporary work files, .log, .pyz and etc. (default: ./build)

-y, --noconfirm Replace output directory (default: SPECPATH/dist/SPECNAME) without asking for confirmation

--upx-dir UPX_DIR Path to UPX utility (default: search the execution path)

--clean Clean PyInstaller cache and remove temporary files before building.

--log-level LEVEL Amount of detail in build-time console messages. LEVEL may be one of TRACE, DEBUG, INFO, WARN, DEPRECATION, ERROR, FATAL (default: INFO). Also settable via and overrides the PYI_LOG_LEVEL environment variable.

What To Generate

-D, --onedir Create a one-folder bundle containing an executable (default)

-F, --onefile Create a one-file bundled executable.

--specpath DIR Folder to store the generated spec file (default: current directory)

-n, --name NAME Name to assign to the bundled app and spec file (default: first script's basename)

--contents-directory CONTENTS_DIRECTORY For onedir builds only, specify the name of the directory in which all supporting files (i.e. everything except the executable itself) will be placed in. Use "." to re-enable old onedir layout without contents directory.

What To Bundle, Where To Search

–add-data SOURCE:DEST

Additional data files or directories containing data files to be added to the application. The argument value should be in form of "source:dest_dir", where source is the path to file (or directory) to be collected, dest_dir is the destination directory relative to the top-level application directory, and both paths are separated by a colon (:). To put a file in the top-level application directory, use . as a dest_dir. This option can be used multiple times.

–add-binary SOURCE:DEST

Additional binary files to be added to the executable. See the `--add-data` option for the format. This option can be used multiple times.

-p, --paths DIR A path to search for imports (like using PYTHONPATH). Multiple paths are allowed, separated by `':'`, or use this option multiple times. Equivalent to supplying the `pathex` argument in the spec file.

--hidden-import, --hiddenimport MODULENAME Name an import not visible in the code of the script(s). This option can be used multiple times.

--collect-submodules MODULENAME Collect all submodules from the specified package or module. This option can be used multiple times.

- Lorenzo Mancini - PyXML import hook fixes under Windows. OS X support. App bundle creation on OS X. Tkinter on OS X. Precompiled bootloaders for OS X.
- Lorenzo Berni - django import hook.
- Louai Al-Khanji - fixes with optparse module.
- Thomas Heller - set custom icon of Windows exe files.
- Eugene Prigorodov <eprigorodov at naumen dot ru> - KInterasDB import hook.
- David C. Morrill - vtkpython import hook.
- Alan James Salmoni - Tkinter interface to PyInstaller.

2.18 Man Pages

2.18.1 pyinstaller

SYNOPSIS

`pyinstaller` <options> SCRIPT…

`pyinstaller` <options> SPECFILE

DESCRIPTION

PyInstaller is a program that freezes (packages) Python programs into stand-alone executables, under Windows, GNU/Linux, macOS, FreeBSD, OpenBSD, Solaris and AIX. Its main advantages over similar tools are that PyInstaller works with Python 3.8-3.11, it builds smaller executables thanks to transparent compression, it is fully multi-platform, and use the OS support to load the dynamic libraries, thus ensuring full compatibility.

You may either pass one or more file-names of Python scripts or a single *.spec*-file-name. In the first case, `pyinstaller` will generate a *.spec*-file (as `pyi-makespec` would do) and immediately process it.

If you pass a *.spec*-file, this will be processed and most options given on the command-line will have no effect. Please see the PyInstaller Manual for more information.

OPTIONS

Positional Arguments

scriptname

Name of scriptfiles to be processed or exactly one .spec file. If a .spec file is specified, most options are unnecessary and are ignored.